COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/31/2026	**DATE SUBMITTED:**	7/31/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	ENGR.JIM LORD QUEJADO

ACTIVITY NO. 4.1 **STACKS**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOB2

Screenshot a code

```
ILOB2 > main.cpp > main()
#include<iostream>
#include "stackList.h"

int main(){

    std::cout<< "--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout<<"\n ------- Popping All the Elements ----- \n";

    while(isEmpty<char>()){
        std::cout<<" Popped: "<< pop<char>() <<std::endl;

    }

    std::cout<<"\n ------ TEsting the Underflow ----- \n";
    pop<char>();

    return 0;


}
```

ANALYSIS:

This is a code that performs some operations on the basic Stack Data Structure including Push, Top, Display, Pop, and Underflow using a templated generic function design that can be found in "stackList.h". The letters c, p, e are pushed into the stack such that e becomes the topmost element of the stack by the LIFO rule. The code is able to print the top element as well as display all elements in a stack from the top to bottom order. The program uses a while loop in order to keep popping the elements from the stack and display them till the stack is emptied out. It turns out that when checking for the underflow of an empty stack, the output "stack overflow." appears.

```
------ TEsting the Underflow -----
PS C:\Users\TIPQC\Documents\CPE010_HOA3_Almirol Joel a>  & 'c:\Use
.4fb' '--stdout=Microsoft-MIEngine-Out-4whcdiax.okd' '--stderr=Mic
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of the stack: e
Displaying the stack:
e
p
c


------- Popping All the Elements -----
Popped: e
Popped: p
Popped: c

------ TEsting the Underflow -----
stack overflow.
```

```
ILOB2 > C stackList.h > Top<T>()
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node {
    public:
    T data; // VALUE THAT WE NEED TO STORE
    Node* next; //PRINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node <T> *head = nullptr;

//operations for the stack

//push
template <typename T>
void push(T newData) {
    //create a new node and assign the newData
    Node <T>* newNode = new Node <T>;
    newNode->data = newData;

    //new node points to current top, then becomes the new top
    newNode->next = head <T>;
    head <T> = newNode;
}
//pop
template <typename T>
T pop() {
    //check if the stack is empty
    if(head <T> == nullptr) {
        std::cout << "stack overflow. " << std::endl;
    }

    //save the current top node
    Node <T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    //move the head to the next node
    head <T> = head <T>->next;
    //free the removed data
    delete temp;
    //return the removed value
    return tempVal;
}
```

ANALYSIS:

The stackList.h offers an implementation of a generic Stack object using a linked list data structure in code through templates, with necessary operations such as push, pop, isEmpty, Top, and displayAll available. A Node<T> is used for each node dynamically allocated, with the global variable of head<T> pointing to the top of the stack to make sure the Last-In, First-Out lifo principle is satisfied. Though the general idea of dynamic allocation of nodes, modification of pointers, and the deletion process (delete temp) works properly, there are two bugs in the code: one is that "stack overflow" appears in the pop() method when an underflow happens (thus explaining why the console shows "stack overflow" the other bug is the lack of return statement in the pop() method when empty.

```
49
50  //isEmpty
51  template <typename T>
52  bool isEmpty() {
53      return head <T> == nullptr;
54  }
55
56  //top
57  template <typename T>
58  void Top() {
59      //error checking
60      if(isEmpty <T>()) {
61          std::cout << "Stack underflow. " << std::endl;
62          return;
63      }
64      //display the top node
65      std::cout << "Top of the stack: " << head<T>->data << std::endl;
66  }
67
68  //display all
69  template <typename T>
70  void displayALL() {
71      if(head <T> == nullptr) {
72          std::cout << "stack underflow. \n";
73          return;
74      }
75
76      std::cout << "Displaying the stack: \n";
77      Node <T>* temp = head <T>;
78      while(temp != nullptr) {
79          std::cout << temp->data << "\n";
80          temp = temp->next;
81      }
82
83      std::cout << "\n";
84  }
85  #endif
```

```
.dk4' '--stdout=Microsoft-MIEngine-Out-g5nyfydb.am0' '--s
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of the stack: e
Displaying the stack:
e
p
c


 ------- Popping All the Elements -----
 Popped: e
 Popped: p
 Popped: c

 ------ TEsting the Underflow -----
stack overflow.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {
    a) If the character read is not a symbol to be balanced, ignore it.
    b) If the character is an opening symbol, push it onto the stack.
    c) If it is a closing symbol:
    i) Report an error if the stack is empty.
    ii) Otherwise, pop the stack.
    d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Self-Checking:

For the following cases, complete the table using the code you created.

HOA3/ILO.C.cpp

```cpp
#include <iostream>
#include <string>
#include <unordered_map>
#include <memory>

using namespace std;

const unordered_map<char, char> BRACKETS = {
    {')', '('},
    {'}', '{'},
    {']', '['}
};

bool isBracket(char ch) {
    return ch == '(' || ch == '{' || ch == '[' ||
           ch == ')' || ch == '}' || ch == ']';
}

bool isOpen(char ch) {
    return ch == '(' || ch == '{' || ch == '[';
}

template <typename T, int SIZE = 100>
class ArrayStack {
private:
    T arr[SIZE];
    int top = -1;

```

```
25  private:
29  public:
30      bool push(T value) {
31          if (top == SIZE - 1) return false;
32          arr[++top] = value;
33          return true;
34      }
35
36      bool pop() {
37          if (empty()) return false;
38          top--;
39          return true;
40      }
41
42      T peek() const {
43          return arr[top];
44      }
45
46      bool empty() const {
47          return top < 0;
48      }
49  };
50
51  template <typename T>
52  class LinkedListStack {
53  private:
54      struct Node {
55          T value;
```

```
    private:
        struct Node {
            unique_ptr<Node> next;
            Node(T v, unique_ptr<Node> n)
                : value(v), next(move(n)) {}
        };

        unique_ptr<Node> head;

    public:
        void push(T value) {
            head = make_unique<Node>(value, move(head));
        }

        bool pop() {
            if (empty()) return false;
            head = move(head->next);
            return true;
        }

        T peek() const {
            return head->value;
        }

        bool empty() const {
            return head == nullptr;
        }
    };
```

```cpp
template <typename Stack>
bool checkBalanced(const string& expr) {
    Stack stack;

    for (char ch : expr) {
        if (!isBracket(ch)) continue;

        if (isOpen(ch)) {
            stack.push(ch);
        } else {
            if (stack.empty()) return false;
            if (stack.peek() != BRACKETS.at(ch)) return false;
            stack.pop();
        }
    }

    return stack.empty();
}

template <typename Stack>
void runTests(const string tests[], int count, const string& title) {
    cout << title << endl;
    for (int i = 0; i < count; i++) {
        cout << tests[i] << " -> "
             << (checkBalanced<Stack>(tests[i]) ? "VALID (Y)" : "INVALID (N)")
             << endl;
    }
```

```cpp
}

int main() {
    string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };

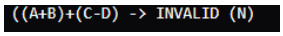
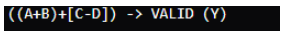
    int total = sizeof(expressions) / sizeof(expressions[0]);
```

```
119
120     int total = sizeof(expressions) / sizeof(expressions[0]);
121
122     runTests<ArrayStack<char>>(expressions, total, "--- Testing Array Stack ---");
123     cout << endl;
124     runTests<LinkedListStack<char>>(expressions, total, "--- Testing Linked List Stack ---");
125
126     return 0;
127 }
```

```
--- Testing Array Stack ---
(A+B)+(C-D) -> VALID (Y)
((A+B)+(C-D) -> INVALID (N)
((A+B)+[C-D]) -> VALID (Y)
((A+B]+[C-D]} -> INVALID (N)

--- Testing Linked List Stack ---
(A+B)+(C-D) -> VALID (Y)
((A+B)+(C-D) -> INVALID (N)
((A+B)+[C-D]) -> VALID (Y)
((A+B]+[C-D]} -> INVALID (N)
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	VALID (Y)	`(A+B)+(C-D) -> VALID (Y)`	Read (Push (onto stack. Stack: ['('] Read) Pop (. Stack empty. Read (Push (onto stack. Stack: ['(']. Read) Pop (. Stack empty. The expression is balanced since every '(' character is pushed into the stack, and the corresponding ')' pops it out from the stack, thus

			ending up in an empty stack at the end of the input.
((A+B)+(C-D)	INVALID (N)		Read (Push (. Stack: ['(']. Read(Push (. Stack: ['(', '(']. Read) Pop (. Stack: ['(']. Read (Push (. Stack: ['(', '(']. Read) Pop (. Stack: ['(']. Expression is invalid because there is one outer opening parenthesis that remains on the stack even after the processing of input has ended, hence the stack is not empty when the input has been processed. The algorithm has failed to meet the condition of reaching the end of the input but finding an empty stack.
(A+B)+[C-D])	VALID (Y)		Read (Push (. Stack: ['(']. Read (Push (. Stack: ['(', '(']. Read) Pop (. Stack: ['(']. Read [Push [. Stack: ['(', '[']. Read] Pop [. Stack: ['(']. Read) Pop (. Stack empty. This expression is correct since each of the opening delimiters in parentheses (and [is correctly paired with the closing delimiters

			) and] in Last-In, First-Out order.
(A+B]+[C-D]}	INVALID (N)	((A+B]+[C-D]} -> INVALID (N)	Read (Push (. Stack: ['(']. Read (Push (. Stack: ['(', '(']. Read]Top of stack is (, which does **not** match closing square bracket]. A mismatched delimiter error is immediately triggered. This string cannot be parsed because when] is encountered and the top of the stack is (, it is a case of unbalanced parentheses, whereas there is no corresponding { for the closing curly brace }.

CONCLUSION

This laboratory activity demonstrates the fundamental role of the Stack data structure in memory management and algorithm design using both Array and Linked List implementations . In the first experiment, one gets to learn about the functionality of LIFO operations and the important exceptions that can happen during their application, such as stack underflow which led to a wrong message "stack overflow" displayed because of incorrect error handling and an unreturned value after popping an empty stack. In the second experiment, the application of stack operations helps one to solve a practical issue faced by compilers, which is checking if symbols in mathematical expressions are matched and closed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.